

QualiMind: A Reproducible Preprocessing Platform for Qualitative and Mixed-Type Data

Final Scientific Project Report (High Integrity Systems, Winter 2025/26)

Kamal Sharma Varshitha Ramamurthy
Ronishabahen Sureshbhai Desai

Kanan Nileshbhai Anadkat
Deepak Kumar

January 2026

Abstract

QualiMind is an intelligent preprocessing platform for qualitative and mixed-type datasets. The system integrates (i) a React-based frontend for dataset upload, preprocessing configuration, and job monitoring, (ii) an Express/TypeScript backend providing strict validation and orchestration, and (iii) an R-Plumber engine that executes preprocessing and optional correlation analysis while generating traceable metadata. The design emphasizes reproducibility through versioned, explicit configuration steps and auditability through column-level transformation logs and encoding/scaling statistics. Processed results are stored in Redis (low-latency retrieval with TTL) and summarized in PostgreSQL (persistent reporting), enabling both interactive analysis and durable provenance. QualiMind is containerized via multi-environment Docker Compose and deployed in a predeployment environment on AWS EC2 behind an Nginx reverse proxy with TLS. This report documents requirements, architecture, preprocessing methodology, operational aspects, and a QA-driven evaluation protocol aligned with the course constraints and deliverables.

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Project Context and Course Constraints	4
1.3	Objectives	4
1.4	Key Contributions	4
2	System Scope and Requirements	4
2.1	Scope	4
2.2	Functional Requirements	5
2.3	Non-Functional Requirements	5
2.4	Course Deliverables Mapping	5
3	Methodology and Project Management (SCRUM)	6
3.1	SCRUM Process and Sprint Cadence	6
3.2	Roles and Rotation	6
4	System Overview	6
4.1	Component Decomposition	6
4.2	Architecture Overview	7
4.3	Architectural Style Justification	7

5	Directory Structure and Codebase Organization	8
5.1	Repository Layout	8
6	Data Flow and Processing Lifecycle	8
6.1	High-Level Lifecycle	8
6.2	Logical Data Flow	9
6.3	Design Rationale: Inline Data vs File-Path Fallback	9
7	Input Data Handling	9
7.1	Supported Formats and Constraints	9
7.2	Parsing Strategy	9
7.3	Normalization to CSV	9
8	Preprocessing Design	10
8.1	Two Execution Modes	10
8.1.1	Tasks Mode (Legacy)	10
8.1.2	Config Mode (Recommended)	10
8.2	Configuration Schema	10
8.3	Pipeline Diagram	11
8.4	Detailed Operations	11
8.4.1	Preflight Steps	11
8.4.2	Missing Values	11
8.4.3	Label Cleaning	11
8.4.4	Reduce Cardinality	12
8.4.5	Encoding	12
8.4.6	Scaling	12
9	Correlation Analysis (Optional)	12
9.1	Configuration	12
9.2	Behavior and Safety Controls	12
10	Metadata Model and Auditability	12
10.1	Metadata Fields	12
10.2	Why Metadata Matters	13
11	Backend Architecture (Software Engineering View)	13
11.1	API Boundaries	13
11.2	Job Orchestration	13
11.3	Realtime Updates (Socket.IO + Redis)	14
11.4	Result Handling and Export	14
12	Frontend Architecture and User Interfaces	14
12.1	UI Inventory (Compliance)	14
12.2	Frontend Configuration Principles	15
13	Database, Storage, and Persistence	15
13.1	Core Entities	15
13.2	Storage Tiers	15
14	Security and Compliance	15
14.1	Security Controls	15
14.2	Compliance Considerations	16

15 DevOps and Deployment	16
15.1 Docker Compose Environments	16
15.2 AWS EC2 Predeployment and Reverse Proxy	16
15.3 Operational Limits	16
16 Testing, Validation, and Quality Assurance	17
16.1 Validation Strategy	17
16.2 Concrete Test Cases	17
16.3 Observability	17
17 Evaluation	17
17.1 Evaluation Protocol	17
17.2 Computational Hotspots	18
17.3 Threats to Validity	18
18 Discussion	18
18.1 Design Trade-offs	18
18.2 Operational Risks	18
18.3 Future Work	18
19 Conclusion	18
A Appendix A: Preprocessing Config Cookbook	19
A.1 Survey Feedback (Categorical + Numeric)	19
B Appendix B: Correlation Config	19
C Appendix C: Auth and Insights API Examples	19

1 Introduction

1.1 Motivation

Real-world datasets used by analysts and data scientists frequently include qualitative attributes (survey answers, categories, labels, IDs) and mixed-type columns (e.g., numeric values stored as strings, partially missing columns, inconsistent tokens such as **NA/NULL**). These issues reduce statistical validity, hinder feature engineering, and introduce instability into machine learning pipelines. A robust preprocessing platform must therefore provide *(i)* deterministic and repeatable transformations, *(ii)* strict configuration to avoid hidden defaults, and *(iii)* traceable metadata so stakeholders can interpret and audit transformations.

1.2 Project Context and Course Constraints

This system was developed in the High Integrity Systems project context (Winter 2025/26). The project specification imposes hard constraints: the solution must include both back-end and front-end implementations; R must be used for data analysis tasks; the system must provide at least four non-trivial user interfaces beyond CRUD and login; the architecture must be well-justified with wireframes and mockups; and Jira must be used for continuous project tracking with Scrum role rotation across sprints.

1.3 Objectives

QualiMind aims to:

- provide a **reproducible preprocessing pipeline** for qualitative and mixed-type datasets;
- expose **strict configuration** (task list mode and step-based config mode);
- generate **traceable metadata** for downstream auditing and interpretability;
- support **operational deployment** (containerization, environments, reverse proxy);
- support analyst workflows via **interactive UI** and export endpoints.

1.4 Key Contributions

- A **config-based preprocessing schema** (versioned, ordered steps, explicit scope).
- An **R-Plumber preprocessing engine** implementing preflight invariants and producing metadata.
- A **backend orchestration layer** with queue-based execution (BullMQ) and realtime updates.
- A **frontend wizard + visualization** enabling configuration, monitoring, and comparison.
- A **DevOps topology** using Docker Compose environments and AWS predeployment routing.

2 System Scope and Requirements

2.1 Scope

QualiMind covers:

- dataset ingestion (CSV/JSON/TXT/XLS/XLSX), normalization, and parsing;

- preprocessing execution in tasks mode and config mode (primary focus);
- optional correlation analysis (numeric-only, post-preprocessing);
- result storage in Redis (fast) and PostgreSQL (persistent summaries);
- job monitoring and export of processed datasets.

Out of scope: model training, automated PII redaction, and automated governance beyond basic access control.

2.2 Functional Requirements

Table 1: Functional requirements

FR-ID	Requirement
FR1	Upload datasets (CSV/JSON/TXT/XLS/XLSX) with MIME filtering and size limit.
FR2	Normalize non-CSV uploads to CSV server-side while preserving original metadata.
FR3	Execute preprocessing in <i>tasks mode</i> (strict task list execution) and <i>config mode</i> (strict ordered steps).
FR4	Apply preflight invariants in config mode: missing token normalization and numeric inference always run.
FR5	Perform preprocessing and optional correlation analysis in R (Plumber). :contentReference[oaicite:2]index=2
FR6	Orchestrate jobs asynchronously via queue (BullMQ) and expose job status.
FR7	Persist results to Redis (TTL) and summaries to PostgreSQL; allow export as CSV/JSON/TXT.
FR8	Provide at least four non-trivial UIs beyond CRUD/login. :contentReference[oaicite:3]index=3

2.3 Non-Functional Requirements

- **Reproducibility:** explicit versioned configs, deterministic step execution, stable metadata.
- **Traceability:** column actions, encoding/scaling stats, high-cardinality flags.
- **Reliability:** validation of configurations, bounded timeouts, graceful storage degradation.
- **Security:** JWT-protected endpoints, authenticated realtime channels, file validation.
- **Deployability:** Docker Compose across environments; EC2 + Nginx reverse proxy + TLS.

2.4 Course Deliverables Mapping

The course expects artifacts including personas/scenarios/user stories, wireframes, system architecture, software design diagrams, mockups, CI/CD, and a final report. :contentReference[oaicite:4]index=4 QualiMind maps to these deliverables by:

- defining analyst personas and job-monitoring workflows;
- providing wireframes and mockups for the upload/config/monitor/visualize screens;
- documenting architecture with component, deployment, and data-flow diagrams;

- producing API, schema, and pipeline design documentation;
- using Jira to track sprint progress and role rotation. :contentReference[oaicite:5]index=5

3 Methodology and Project Management (SCRUM)

3.1 SCRUM Process and Sprint Cadence

The project follows SCRUM methodology divided into five sprints with fixed sprint dates (Sprint 1 starts October 27; final delivery in late January).

3.2 Roles and Rotation

Per project constraints, each sprint must nominate a Team Captain (Scrum Master) and a Product Owner, both rotated so each member performs each role once, documented in deliverables. :contentReference[oaicite:7]index=7 Table 2 documents the team responsibilities as implemented.

Table 2: Team responsibilities (project-level)

Member	Primary Responsibilities
Kamal Sharma	Backend development, CI/CD, frontend lead, R integration lead
Varshitha Ramamurthy	Documentation lead, UI/UX lead
Kanan N. Anadkat	Frontend support, UI/UX support
Ronishababen S. Desai	QA support, frontend support
Deepak Kumar	QA testing lead, R integration support

4 System Overview

4.1 Component Decomposition

QualiMind is composed of the following subsystems:

- **Frontend (React):** upload, configuration wizard, job monitor, data visualization.
- **Backend (Express + TypeScript):** API validation, dataset/job records, orchestration.
- **Worker (BullMQ):** background preprocessing execution and result persistence.
- **R Engine (Plumber):** preprocessing, correlation, metadata generation, storage summary writes.
- **Redis:** short-lived result cache for fast retrieval and realtime operations.
- **PostgreSQL:** persistent storage for datasets, jobs, and processing summaries.

4.2 Architecture Overview

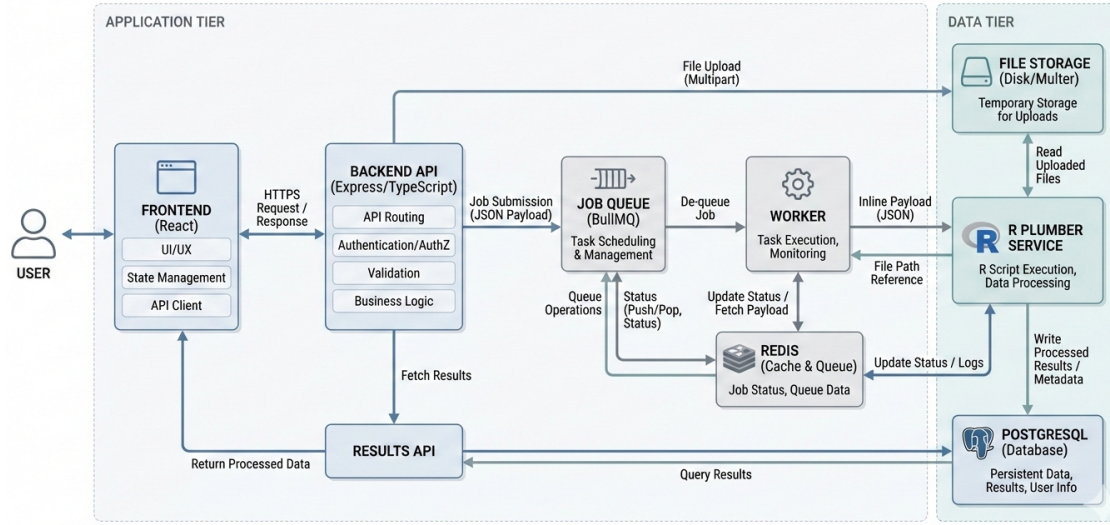


Figure 1: High-level architecture of QualiMind: UI, orchestration backend, worker queue, R engine, and storage tiers.

4.3 Architectural Style Justification

QualiMind adopts a **service-oriented, pipeline-centered architecture**:

- preprocessing logic is isolated in the R engine to ensure analytical correctness and consistency;
- orchestration (validation, job lifecycle, storage policy) is separated into the backend/worker;
- UI concerns (configuration and visualization) are isolated in the frontend;
- this decomposition reduces coupling, enables independent scaling of the R engine, and supports reproducibility by centralizing transformation semantics.

This is aligned with the required “well-justified architectural style” supported by wireframes/-mockups. :contentReference[oaicite:9]index=9

5 Directory Structure and Codebase Organization

5.1 Repository Layout

Table 3: Directory guide

Directory	Contents	Primary Role
<code>r-engine/</code>	R-Plumber service; preprocessing; correlation; storage writes	Data preprocessing core
<code>server/</code>	API; jobs; queue; schemas; storage integration	Orchestration & control plane
<code>frontend/</code>	UI, upload wizard, config builder, monitoring	User-facing interface
<code>datasets/</code>	Sample datasets for testing	Test inputs
<code>infra/</code>	Terraform and deployment assets	Infrastructure
<code>docker-compose.*.yaml</code>	Environment orchestration definitions	Runtime & DevOps

6 Data Flow and Processing Lifecycle

6.1 High-Level Lifecycle

1. User uploads dataset and selects preprocessing configuration.
2. Backend creates `Dataset` and `ProcessingJob` records.
3. Job is enqueued in BullMQ (`preprocess` queue).
4. Worker reads normalized CSV and calls R engine endpoint `/clean`.
5. R engine executes preprocessing (and optional correlation) and returns data + metadata.
6. Backend/worker persist outputs to Redis and PostgreSQL; exports generated as requested.
7. Frontend polls/subscribes and visualizes results (original vs processed).

6.2 Logical Data Flow

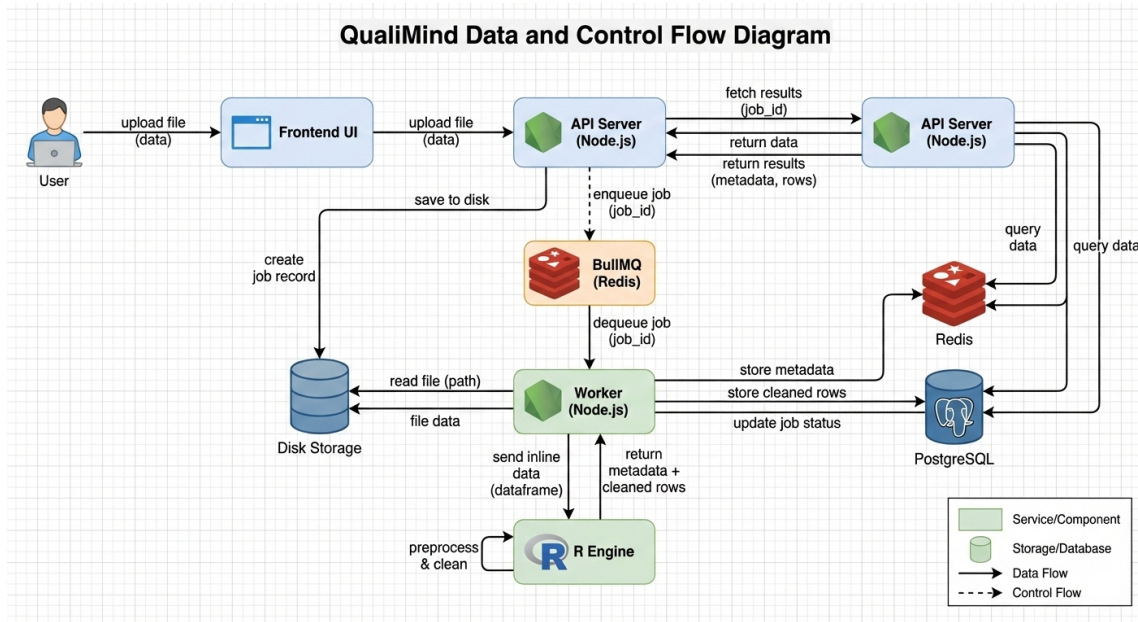


Figure 2: Data flow from upload through orchestration, R preprocessing, storage, and UI retrieval.

6.3 Design Rationale: Inline Data vs File-Path Fallback

The worker parses CSV to structured row records and prefers inline payloads when calling R (reducing filesystem coupling). If required by size or workflow, the R engine can fall back to file-path loading, improving operational flexibility while maintaining a consistent interface.

7 Input Data Handling

7.1 Supported Formats and Constraints

Supported formats: CSV, JSON, TXT, XLS/XLSX. Maximum file size: 20MB. The backend enforces MIME validation and size restrictions, reducing ingestion risk and limiting downstream R memory pressure.

7.2 Parsing Strategy

The backend reads and parses CSV into row records prior to invoking the R engine. This supports:

- consistent structural validation before preprocessing,
- schema inference on the application side (UI previews and type inference),
- stable payload format for R routines.

7.3 Normalization to CSV

Non-CSV uploads (JSON/XLS/XLSX) are converted server-side to a temporary CSV:

- original filename and MIME type are preserved for metadata and auditability,

- worker uses the normalized CSV as the canonical processing input,
- conversion reduces format variance and simplifies downstream logic.

8 Preprocessing Design

8.1 Two Execution Modes

8.1.1 Tasks Mode (Legacy)

Tasks mode accepts a `preprocessingTasks` array and executes only tasks listed (strict, no defaults). This mode is suitable for backwards compatibility and coarse control.

8.1.2 Config Mode (Recommended)

Config mode accepts a versioned `preprocessingConfig` that specifies:

- ordered, explicit preprocessing steps,
- scoped application by column names or semantic types (categorical/numeric),
- method-specific parameters.

A preflight invariant enforces missing-token normalization and numeric type inference before user steps, improving robustness of mixed-type data.

8.2 Configuration Schema

Listing 1: Preprocessing configuration schema (summary)

```
{
  "version": "1.0",
  "steps": [
    {
      "task": "missing_values" | "label_cleaning" | "reduce_cardinality" |
            "encoding" | "scaling" | "missing_token_normalization" |
            "numeric_type_inference",
      "method": "...",
      "appliesTo": { "types": ["categorical" | "numeric"], "columns": ["colA"] },
      "params": { "...": "..." }
    }
  ]
}
```

8.3 Pipeline Diagram

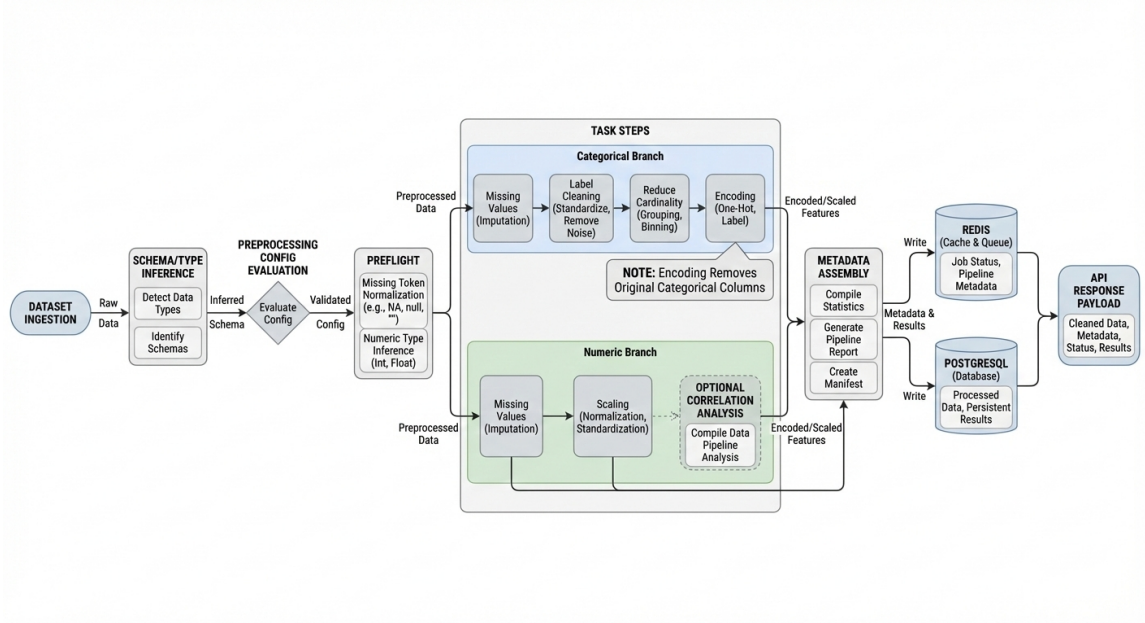


Figure 3: Preprocessing pipeline stages implemented by the R engine in config mode.

8.4 Detailed Operations

8.4.1 Preflight Steps

Missing token normalization trims strings and maps tokens such as NULL, NA, and empty values to NA. This step reduces data-quality heterogeneity introduced by human and system conventions.

Numeric type inference identifies numeric-like strings and converts them to numeric values based on an inference threshold (e.g., 0.9). This ensures that numeric operations (imputation/scaling/correlation) operate on consistent numeric domains.

8.4.2 Missing Values

Methods support numeric and categorical strategies (Table 4).

Table 4: Missing-value handling methods

Method	Type	Behavior	Params
<code>numeric_median</code>	numeric	Replace NA with median	—
<code>numeric_mean</code>	numeric	Replace NA with mean	—
<code>numeric_constant</code>	numeric	Replace NA with constant value	<code>value</code>
<code>categorical_mode</code>	categorical	Replace NA with modal category	—
<code>categorical_unknown</code>	categorical	Replace NA with constant label	<code>unknownLevel</code>

8.4.3 Label Cleaning

Label cleaning standardizes categorical strings by trimming whitespace, lowercasing, removing punctuation, and normalizing underscores. This reduces accidental cardinality growth caused by case/whitespace variations.

8.4.4 Reduce Cardinality

To prevent feature explosion and improve stability:

- rare categories below `rare_prop_threshold` are collapsed into `other`,
- columns exceeding `high_cardinality_threshold` are flagged in metadata for analyst review.

8.4.5 Encoding

QualiMind uses a pragmatic encoding policy:

- frequency encoding always adds `<col>_freq` features,
- one-hot encoding is used when unique levels $\leq$ `one_hot_max_levels`,
- otherwise label encoding is applied with a stable mapping,
- original categorical columns are removed after encoding to avoid leakage and redundancy.

8.4.6 Scaling

Numeric scaling supports `zscore` or `minmax` transformations, or `none` for audit-friendly raw exports. Scaling stats are recorded to metadata for interpretability and QA.

9 Correlation Analysis (Optional)

9.1 Configuration

Listing 2: Correlation configuration

```
{
  "columns": ["colA", "colB"],
  "method": "pearson" | "spearman" | "kendall",
  "topK": 50,
  "minAbs": 0,
  "includeMatrix": true,
  "maxMatrixCols": 200
}
```

9.2 Behavior and Safety Controls

Correlation is computed post-preprocessing and numeric-only. The engine excludes constant or missing columns to avoid invalid correlations, and bounds matrix generation via `maxMatrixCols` to limit quadratic blow-up in memory/time.

10 Metadata Model and Auditability

10.1 Metadata Fields

Key metadata fields include:

- row/column counts before and after processing;
- lists of categorical/numeric columns and final numeric columns;
- encoding outputs (encoded columns, frequency columns, mapping summaries);

- scaling stats (mean, std, min, max depending on method);
- rare-category info and high-cardinality flags;
- `column_actions` for step-by-step traceability (config mode);
- correlation results if enabled.

10.2 Why Metadata Matters

Metadata enables:

- **reproducibility:** rerunning with the same config yields comparable outputs;
- **auditability:** analysts can justify transformations in reports;
- **debuggability:** column actions explain unexpected type/feature changes;
- **governance support:** storage summaries create traceable provenance.

11 Backend Architecture (Software Engineering View)

11.1 API Boundaries

Core APIs:

- `/api/health`: health checks
- `/api/auth`: signup/login, Google OAuth, verification, reset
- `/api/datasets`: upload, list, update, delete
- `/api/jobs`: status, results, export
- `/api/preprocessing/suggest`: LLM-assisted `preprocessingConfig` suggestion
- `/api/insights`: LLM-assisted narrative insights

11.2 Job Orchestration

Jobs are enqueued to BullMQ queue `preprocess`. The worker executes the preprocessing pipeline, persists outputs, and publishes lifecycle updates. This decouples UI interactions from compute-intensive steps and improves robustness under variable processing times.

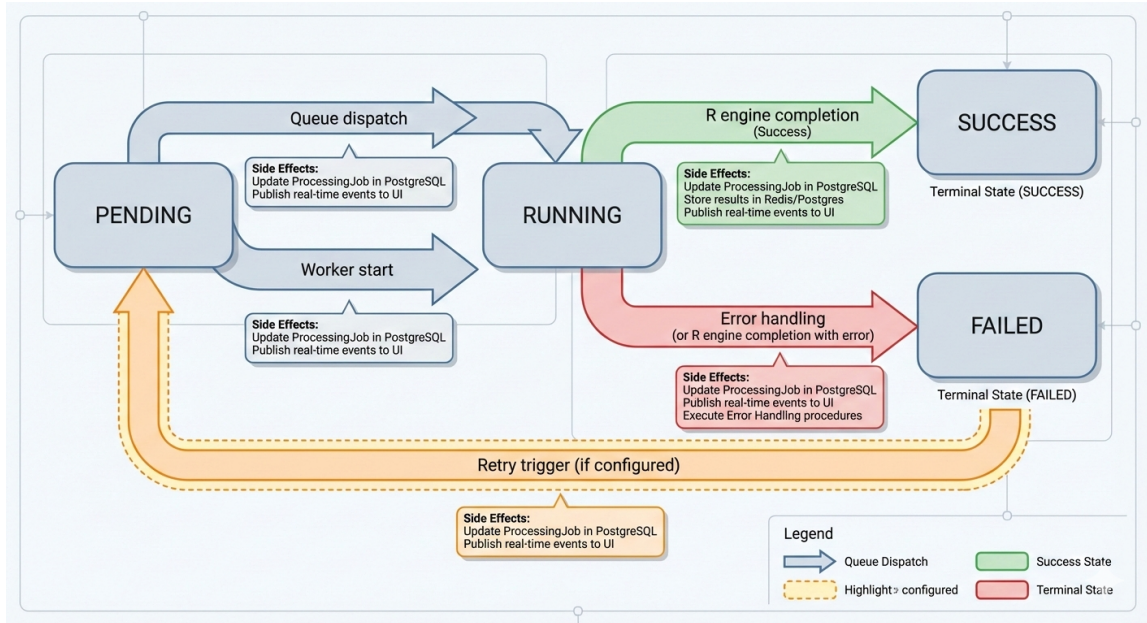


Figure 4: Job state transitions and worker-driven lifecycle reporting.

11.3 Realtime Updates (Socket.IO + Redis)

Worker publishes events to Redis channel `job:updates`; API server subscribes and relays events via Socket.IO to authenticated clients (JWT-based). This provides near real-time feedback on job progress and completion.

11.4 Result Handling and Export

A result store layer persists processed datasets to Redis (for fast retrieval) and to filesystem exports (CSV/JSON/TXT). Durable summary records are stored in PostgreSQL for traceability and reporting.

12 Frontend Architecture and User Interfaces

12.1 UI Inventory (Compliance)

The project requires at least four UIs beyond basic CRUD and login. :contentReference[oaicite:10]index=10 QualiMind includes the following non-trivial UIs:

1. **Dataset Upload Wizard:** file validation, format normalization feedback, preview sampling.
2. **Preprocessing Orchestrator (Step 2):** config building, validation, column scoping controls.
3. **Correlation Orchestrator (Step 3):** column selection, method configuration, matrix constraints.
4. **Job Monitor Dashboard:** lifecycle tracking, realtime updates, export controls.
5. **Dataset Visualization Panel:** original vs processed distributions (categorical breakdown; numeric summaries).

12.2 Frontend Configuration Principles

The frontend builds `preprocessingConfig` from explicit user choices and validates configurations before submission. It also infers column types from sampled rows to support analyst-friendly defaults without introducing implicit preprocessing steps (strictness remains enforced by the backend and R pipeline).

13 Database, Storage, and Persistence

13.1 Core Entities

Key tables include:

- **User:** identity and authentication state
- **Dataset:** ownership, file metadata, upload provenance
- **ProcessingJob:** `preprocessingConfig`, `correlationConfig`, status, timestamps
- **dataset_processing_summary:** summary produced by the R engine

13.2 Storage Tiers

- **Redis:** 24h TTL for processed results and metadata (fast retrieval).
- **PostgreSQL:** persistent summaries and job metadata.
- **Filesystem:** export caching (CSV/JSON/TXT).

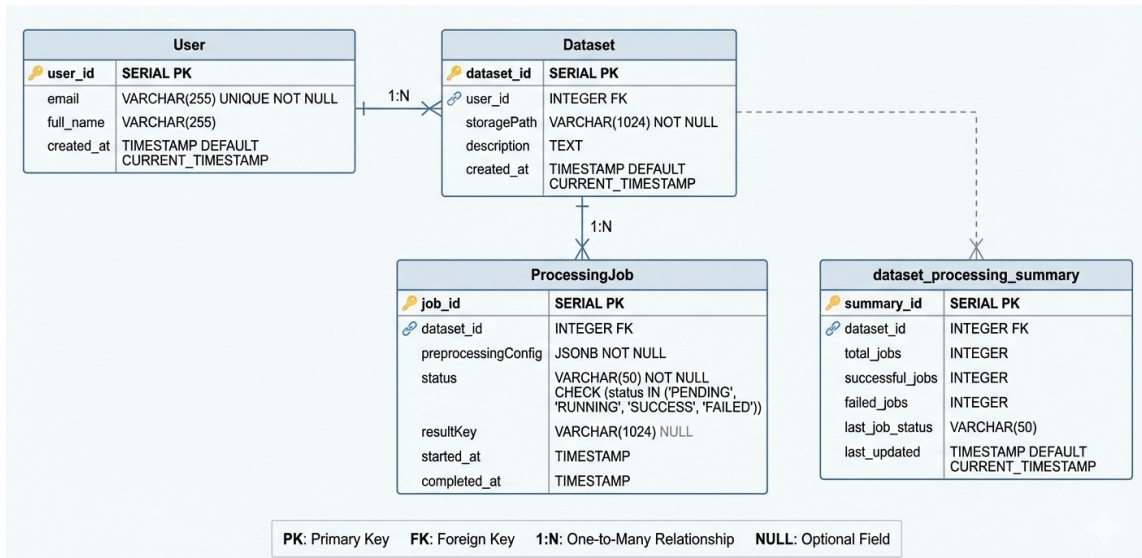


Figure 5: Core schema: datasets, jobs, summaries, and user ownership.

14 Security and Compliance

14.1 Security Controls

- JWT-based authentication for protected API routes.
- Email verification for password signups; Google OAuth support.

- JWT-authenticated Socket.IO handshake for realtime channels.
- File type and file size validation at upload.

14.2 Compliance Considerations

QualiMind does not automatically redact PII. Responsibility remains with analysts to ensure compliance with institutional and legal requirements. This design choice is explicit to avoid silent data modification and to preserve transparency.

15 DevOps and Deployment

15.1 Docker Compose Environments

The system provides separate Docker Compose configurations for:

- development (`docker-compose.dev.yml`),
- testing (`docker-compose.testing.yml`),
- predeploy (`docker-compose.predeploy.yml`).

Services include Postgres, Redis, API server, worker, frontend, and R engine.

15.2 AWS EC2 Predeployment and Reverse Proxy

A predeployment environment runs on AWS EC2. Nginx acts as a reverse proxy with TLS certificates provisioned via Certbot and routes traffic to the public endpoint <https://qualimind.kamalbhاییi.de>.

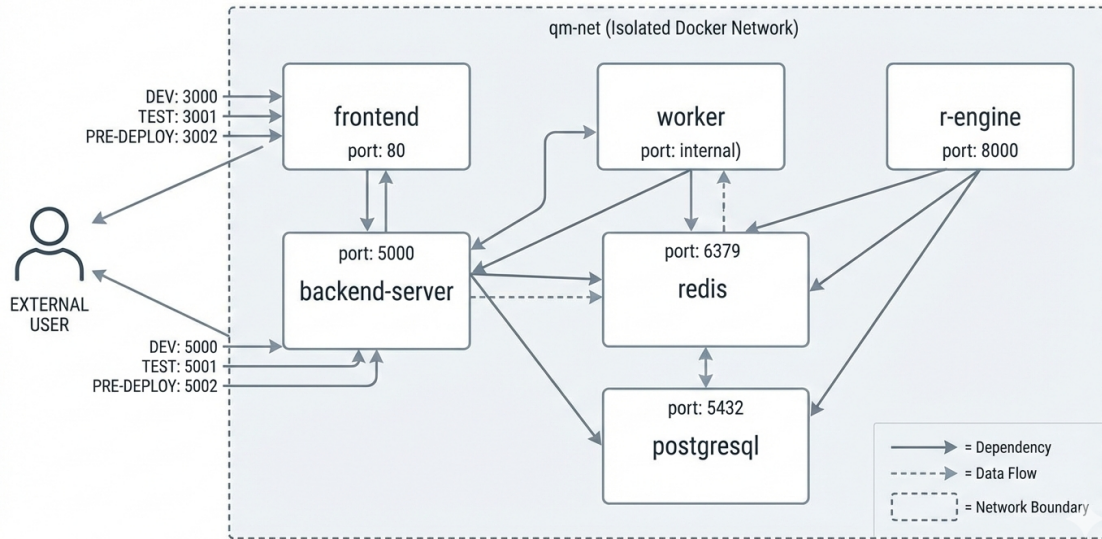


Figure 6: Deployment topology: EC2 hosting services, Nginx reverse proxy, and TLS termination.

15.3 Operational Limits

- file size limit: 20MB;
- R engine timeout: 5 minutes (backend client timeout);

- Redis TTL: 24 hours;
- correlation matrix safety bound: `maxMatrixCols`.

16 Testing, Validation, and Quality Assurance

16.1 Validation Strategy

Validation is layered:

- **API-level schema validation:** invalid `preprocessingConfig` rejected (HTTP 400).
- **Pipeline-level validation:** constant/missing columns excluded from correlation.
- **Storage validation:** ensure consistency across Redis and PostgreSQL summaries.
- **Export validation:** CSV/JSON/TXT outputs match expected formatting and row counts.

16.2 Concrete Test Cases

Table 5: QA test cases (representative)

Case	Input	Expected Outcome
A	Categorical column with NA	NA replaced (mode or unknown); <code>column_actions</code> updated
B	Numeric column with known mean/SD	z-score yields mean ≈ 0 , sd ≈ 1 ; <code>scaling_stats</code> populated
C	Categorical column with ≤ 10 levels	one-hot columns added; original removed; encoding metadata shows method
D	Two+ numeric columns	topK correlations returned; matrix included if within constraints

16.3 Observability

Logging is implemented in backend and worker; health endpoints exist for API and R engine. Recommended future improvements include centralized logging and operational metrics (runtime, failure rates).

17 Evaluation

17.1 Evaluation Protocol

Evaluation focuses on correctness, reproducibility, and operational viability:

- **Correctness:** transformations applied exactly as specified by config steps.
- **Reproducibility:** reruns on identical input/config yield consistent column actions and outputs.
- **Metadata completeness:** required metadata keys populated across typical configs.
- **Performance envelope:** job runtime and memory hotspots recorded for representative datasets.

17.2 Computational Hotspots

- High-cardinality encoding may generate many features (one-hot expansion).
- Correlation is $O(n^2)$ in number of selected numeric columns, hence bounded matrix generation.

17.3 Threats to Validity

- Datasets may not represent all real-world corner cases (e.g., extreme cardinality, nested JSON).
- Preflight type inference may change semantics for some ambiguous columns; mitigated by `column_actions`.
- Correlation computed post-preprocessing; encoding choices can affect correlation interpretability.

18 Discussion

18.1 Design Trade-offs

Strictness vs Convenience: QualiMind explicitly avoids implicit default steps in tasks mode and requires explicit steps in config mode to ensure traceability and reproducibility. This improves scientific auditability but increases configuration effort.

R Engine Isolation: Concentrating preprocessing in R ensures analytical correctness and reusability, while the Node backend focuses on validation, orchestration, and integration.

18.2 Operational Risks

- large datasets may exceed R memory;
- request timeouts can trigger failures on heavy encoding/correlation tasks;
- Redis TTL can drop results if summaries are not persisted or exported.

18.3 Future Work

Planned extensions include advanced NLP preprocessing, RBAC, data-quality scoring, anomaly detection, MLOps integration, and scaling the R engine horizontally with prioritization.

19 Conclusion

QualiMind delivers a reproducible preprocessing platform for qualitative and mixed-type datasets, combining strict configuration, traceable metadata, and operational deployment readiness. The architecture cleanly separates UI, orchestration, and analytical execution; integrates Redis and PostgreSQL for layered persistence; and supports a queue-based execution model with realtime job updates. The system fulfills the course constraints regarding full-stack implementation, use of R for analysis, multiple user interfaces beyond CRUD/login, and requirement-driven documentation artifacts.

A Appendix A: Preprocessing Config Cookbook

A.1 Survey Feedback (Categorical + Numeric)

```
{
  "version": "1.0",
  "steps": [
    { "task": "missing_values", "method": "categorical_unknown",
      "appliesTo": { "types": ["categorical"] },
      "params": { "unknownLevel": "unknown" } },
    { "task": "label_cleaning", "method": "standard",
      "appliesTo": { "types": ["categorical"] } },
    { "task": "reduce_cardinality", "method": "rare_to_other",
      "appliesTo": { "types": ["categorical"] },
      "params": { "rare_prop_threshold": 0.01, "high_cardinality_threshold": 50 } },
    { "task": "encoding", "method": "auto",
      "appliesTo": { "types": ["categorical"] },
      "params": { "one_hot_max_levels": 10 } },
    { "task": "missing_values", "method": "numeric_median",
      "appliesTo": { "types": ["numeric"] } },
    { "task": "scaling", "method": "zscore",
      "appliesTo": { "types": ["numeric"] } }
  ]
}
```

B Appendix B: Correlation Config

```
{
  "columns": ["colA", "colB"],
  "method": "pearson",
  "topK": 50,
  "minAbs": 0,
  "includeMatrix": true,
  "maxMatrixCols": 200
}
```

C Appendix C: Auth and Insights API Examples

Include your cURL examples from the README here if required for submission.

Acknowledgement

This project is developed for the High Integrity Systems course (Winter Semester 2025/26) at Frankfurt University of Applied Sciences. We thank Prof. Dr. Christina Andersson for academic guidance and support.